Project Report

Implement Client Script and UI Policy (Incident)

Date	29 September 2026
Team ID	SWTID-2026-7872
Project Name	Implement Client Script and UI Policy (Incident)

Project Report Format

1. INTRODUCTION
- 1.1 Project Overview
- 1.2 Purpose

2. IDEATION PHASE
- 2.1 Problem Statement
- 2.2 Empathy Map Canvas
- 2.3 Brainstorming

3. REQUIREMENT ANALYSIS
- 3.1 Customer Journey Map
- 3.2 Solution Requirement
- 3.3 Data Flow Diagram
- 3.4 Technology Stack

4. PROJECT DESIGN
- 4.1 Problem Solution Fit
- 4.2 Proposed Solution
- 4.3 Solution Architecture

5. PROJECT PLANNING & SCHEDULING
- 5.1 Project Planning

6. FUNCTIONAL AND PERFORMANCE TESTING
- 6.1 Performance Testing

7. RESULTS
- 7.1 Output Screenshots

8. ADVANTAGES & DISADVANTAGES

9. CONCLUSION

10. FUTURE SCOPE

11. APPENDIX
- Source Code (if any)
- Dataset Link
- GitHub & Project Demo Link

1. INTRODUCTION

1.1 Project Overview

This project implements **Client Scripts** and **UI Policies** on the Incident form of the ServiceNow platform. UI Policies declaratively control field behaviour (mandatory, visible, read-only) based on conditions such as State or Priority, while Client Scripts (onLoad, onChange, onSubmit, onCellEdit) run JavaScript in the browser for validation and dynamic behaviour. Where server data is needed, a client script calls a client-callable Script Include through GlideAjax.

1.2 Purpose

The purpose of the project is to improve the quality and consistency of incident data by validating input at the point of entry. It aims to reduce incomplete or incorrect incident records, guide agents with clear messages, and speed up resolution by ensuring the right information is captured the first time.

2. IDEATION PHASE

2.1 Problem Statement

Service desk agents often save incidents with missing or inconsistent details (for example, no close notes on resolved incidents, or a stale subcategory after the category changes). This leads to rework, wrong routing and delayed resolution. A form that enforces rules at entry time is needed.

PS No.	I am	I am trying to	But	Because	Which makes me feel
PS-1	A service desk agent	Log and resolve incidents quickly and correctly	Required details are often missed or inconsistent	The form does not enforce rules or guide me while I type	Frustrated and slowed down by rework
PS-2	A service desk manager	Rely on incident data for routing and reporting	Records are incomplete or inaccurate	Validation is done manually after the fact	Concerned about service quality

2.2 Empathy Map Canvas

Says	Thinks
"I forgot to add close notes." / "Which subcategory applies now?"	Wants to finish tickets fast without mistakes; worries about reopened incidents.
Does	**Feels**
Fills the form quickly, edits fields repeatedly, fixes errors after submission.	Frustrated by rework, uncertain about required fields, relieved when the form guides them.

Pains: missing mandatory data, confusing fields, errors found late. **Gains:** clear prompts, fewer rejected records, faster resolution.

2.3 Brainstorming

- Make Short description and Caller mandatory when the form loads (UI Policy).
- Require Close code and Close notes when State is Resolved (UI Policy).
- Show resolution fields only when they are needed (UI Policy).
- Clear Subcategory when Category changes (onChange Client Script).
- Warn agents when Priority is set to 1 - Critical (onChange).
- Block submission when the Description is too short (onSubmit).
- Flag VIP callers using GlideAjax and a Script Include.

Prioritisation: High - mandatory-field policies, Category/Subcategory handling and onSubmit validation. Medium - VIP check and field-visibility rules. Low - onCellEdit list validation.

3. REQUIREMENT ANALYSIS

3.1 Customer Journey Map

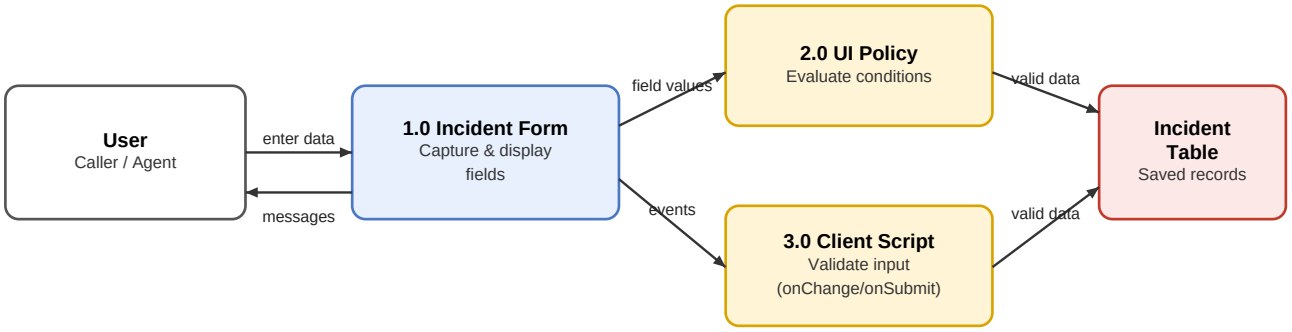
Stage	Agent Action	Touchpoint	Experience	Opportunity
Open	Opens a new Incident	Incident form	Neutral	UI Policy marks required fields at once
Fill	Enters Caller, Category and details	Form fields	May be unsure	onChange scripts guide and clear stale values
Submit	Clicks Submit	Form	Anxious about errors	onSubmit script catches problems before saving
Resolve	Sets State to Resolved	State, resolution fields	Often forgets notes	Close code/notes become mandatory
Close	Record saved	Incident table	Satisfied	Clean, complete data for reporting

3.2 Solution Requirement

FR No.	Functional Requirement (Epic)	Sub Requirement (Story / Sub-Task)
FR-1	UI Policy	Mandatory, visible and read-only behaviour driven by conditions
FR-2	Client Script - onLoad / onChange	Info messages, clearing dependent fields, priority warning, caller auto-fill
FR-3	Client Script - onSubmit / onCellEdit	Block invalid saves and validate list edits
FR-4	GlideAjax and Script Include	Server look-up (e.g., VIP caller, open incident count)
FR-5	Testing and Deployment	Test cases, update set migration, UAT sign-off

NFR No.	Non-Functional Requirement	Description
NFR-1	Usability	Clear, consistent messages shown on the form
NFR-2	Security	Client-side checks supported by server-side enforcement; role-based access respected
NFR-3	Reliability	Rules behave the same across desktop, mobile and Service Portal UI types
NFR-4	Performance	Incident form loads in under 3 seconds with all scripts active
NFR-5	Availability	Changes captured in update sets for repeatable deployment
NFR-6	Scalability	Scripts are lightweight and avoid unnecessary server calls

3.3 Data Flow Diagram



Errors / mandatory prompts flow back to the user through the form (g_form messages).

Figure 1: Data flow of user input through the Incident form, UI Policy and Client Script

3.4 Technology Stack

Component	Technology
Platform	ServiceNow (Personal Developer Instance)
Client-side logic	Client Scripts (JavaScript), UI Policies, g_form API
Server-side logic	Script Include (Client Callable), GlideRecord
Async communication	GlideAjax
Deployment	Update Sets
Testing	Browser testing, UAT test cases

4. PROJECT DESIGN

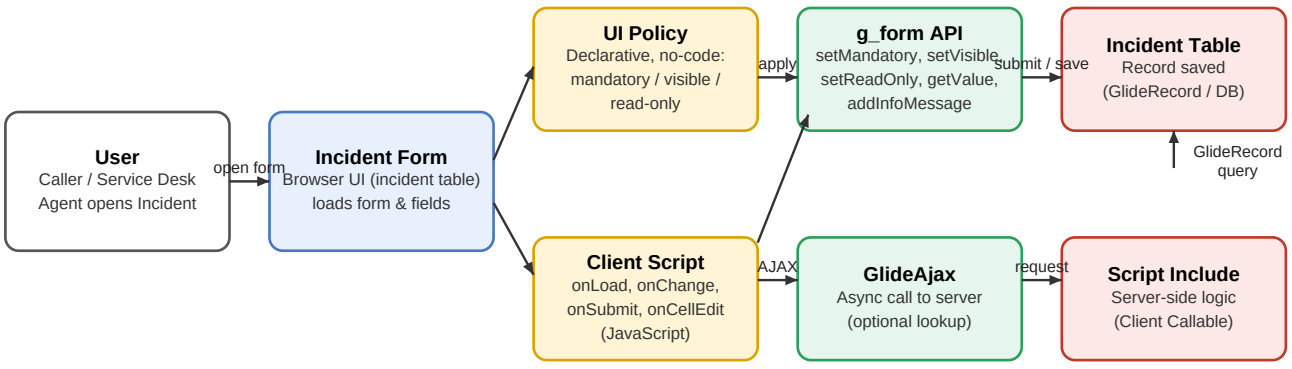
4.1 Problem Solution Fit

The problem is incomplete or inconsistent incident data caused by entry mistakes. Validating at the form, before the record is saved, directly removes the cause. UI Policies handle simple rules without code, and Client Scripts handle logic that needs JavaScript, so each rule uses the lightest suitable tool.

4.2 Proposed Solution

S.No.	Parameter	Description
1	Problem Statement	Incident records are saved with missing or inconsistent data
2	Idea / Solution	UI Policies and Client Scripts validate and guide input on the Incident form
3	Novelty	Combines no-code UI Policies with targeted scripts and a GlideAjax server look-up
4	Social Impact / Customer Satisfaction	Faster, more accurate resolution; fewer reopened tickets
5	Business Model	Improves service-desk efficiency and data quality on an existing platform
6	Scalability	Same pattern can be extended to other tables and forms

4.3 Solution Architecture



Flow A: UI Policy sets field behaviour on form load / field change (no code)

Flow B: Client Script runs custom logic; may call Script Include via GlideAjax

Figure 2: Architecture and data flow of Client Script and UI Policy implementation on the Incident form (ServiceNow)

5. PROJECT PLANNING & SCHEDULING

5.1 Project Planning

Sprint	Epic	User Stories	Story Points	Start Date	End Date (Planned)
Sprint-1	UI Policy (Incident)	USN-1 to USN-8	20	05 Oct 2026	10 Oct 2026
Sprint-2	Client Script (onLoad / onChange)	USN-9 to USN-15	20	12 Oct 2026	17 Oct 2026
Sprint-3	Client Script (onSubmit / GlideAjax)	USN-16 to USN-21	20	19 Oct 2026	24 Oct 2026
Sprint-4	Testing & Deployment	USN-22 to USN-27	20	26 Oct 2026	31 Oct 2026

Total scope is 80 story points over four 6-day sprints. Average velocity = 20 / 6 ≈ 3.33 story points per day.

6. FUNCTIONAL AND PERFORMANCE TESTING

6.1 Performance Testing

Test Case ID	Scenario	Expected Result	Actual Result	Pass/ Fail
FT-01	Mandatory fields via UI Policy	Form not saved; Caller and Short description mandatory		
FT-02	Conditional mandatory fields	Close code/notes mandatory when State is Resolved		
FT-03	onChange Client Script	Subcategory cleared when Category changes		
FT-04	onSubmit Client Script	Submit blocked for Description under 20 characters		
PT-01	Form load time	Under 3 seconds		
PT-02	GlideAjax response	Script Include responds without slowing the form		
PT-03	Concurrent form load	Multiple forms work without errors		

Actual results and pass/fail values are recorded during test execution (see the Functional & Performance Testing and UAT documents).

7. RESULTS

7.1 Output Screenshots

Incident form with mandatory fields highlighted (UI Policy)

[Insert screenshot]

Resolution fields shown/hidden based on State

[Insert screenshot]

onChange message after changing Category

[Insert screenshot]

onSubmit error message for a short Description

[Insert screenshot]

GlideAjax VIP caller warning

[Insert screenshot]

8. ADVANTAGES & DISADVANTAGES

Advantages	Disadvantages
Better data quality at the point of entry	Client-side rules can be bypassed by integrations or imports, so server-side enforcement is also needed
Fewer incomplete records and less rework	Too many scripts can slow form load
UI Policies need no code and are easy to maintain	Conflicts between UI Policies and Client Scripts need careful ordering and testing
Clear guidance for agents through messages	Requires ongoing maintenance as processes change

9. CONCLUSION

The project shows how Client Scripts and UI Policies on the ServiceNow Incident form improve data quality and guide users while they work. UI Policies cover declarative rules and Client Scripts cover logic and server look-ups, together giving a form that is more reliable and easier to use.

10. FUTURE SCOPE

- Extend the same validations to Problem, Change and Request forms.
- Add matching server-side Business Rules and Data Policies for imports and integrations.
- Use Catalog Client Scripts for Service Catalog items.
- Add dashboards that track data-quality improvement.

11. APPENDIX

Source Code (if any)

Sample client script - onChange (Category):

```
function onChange(control, oldValue, newValue, isLoading, isTemplate) {
    if (isLoading || newValue === '') { return; }
    g_form.clearValue('subcategory');
    g_form.addInfoMessage('Category changed - select a new Subcategory.');
}
```

Sample client script - onSubmit (Description length):

```
function onSubmit() {
    var desc = g_form.getValue('description');
    if (desc.length < 20) {
        g_form.showFieldMsg('description', 'Enter at least 20 characters.', 'error');
        return false;
    }
}
```

Sample UI Policy: Table = Incident; Condition = State is Resolved; Actions = Close code (Mandatory: True), Close notes (Mandatory: True); Reverse if false = checked.

Dataset Link

Not applicable - this project configures an existing ServiceNow table and does not use an external dataset.

GitHub & Project Demo Link

GitHub: [add repository link]
Demo video: [add demo link]